

Вінницький національний технічний університет _____

(повне найменування вищого навчального закладу)

Кафедра _____

комп'ютерних

наук _____

(повна назва кафедри)

КУРСОВА РОБОТА

з дисципліни «Алгоритмізація та програмування»

(назва дисципліни)

на тему: «Розробка серверної частини інформаційної системи VocabMiner»

Студента 1 курсу, групи 4КН-25Б

Спеціальності 122 Комп'ютерні науки

Саволук Микола Миколайович

(прізвище, ім'я, по батькові)

Керівник асистент кафедри комп'ютерних наук, аспірант Кудрявцев Дмитро
Станіславович

(посада, науковий ступінь, прізвище та ініціали)

Національна шкала _____

Кількість балів: _____ Оцінка: ECTS _____

Члени комісії _____

(підпис) (прізвище та ініціали)

(підпис) (прізвище та ініціали)

м. Вінниця - 2026 рік

Міністерство освіти і науки України

Вінницький національний технічний університет

Факультет інформаційних технологій та комп'ютерної інженерії

Кафедра комп'ютерних наук

ЗАТВЕРДЖУЮ

Зав. каф. КН _____

«__» _____ 2026 р.

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

до курсової роботи з дисципліни «Алгоритмізація та програмування»

Студента Саволюка Миколи Миколайовича групи 4КН-25Б

Тема курсової роботи: «Розробка серверної частини інформаційної системи VocabMiner», затверджена на засіданні кафедри комп'ютерних наук, протокол № ____ від «__» _____ 2026 р.

Вихідні дані:

1. Тип системи: серверна частина інформаційної системи для майнінгу словника з навчального контенту.
2. Архітектура: клієнт-серверна, з поділом на контролери, бізнес-логіку, репозиторії та доменні моделі.
3. База даних: PostgreSQL.
4. Технологія реалізації: ASP.NET Core 8, Entity Framework Core, Npgsql.
5. Основні сутності: користувачі, джерела контенту, сегменти, словникові одиниці, появи словникових одиниць, спроби повторення та завдання експорту.
6. Мінімальні вимоги: не менше 3 сутностей із 5 і більше полями, CRUD-операції, 5-8 контролерів, ViewModels, сервіси, репозиторії, dependency injection.
7. Середовище розробки: .NET 8 SDK, Docker, PostgreSQL.
8. Перевірка працездатності: unit-тести, HTTP-сценарій і live-перевірка з реальною PostgreSQL БД.

ЗМІСТ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ

Титульний лист.

Завдання на виконання.

Анотація.

Зміст.

Вступ.

1. Аналіз предметної області та постановка задачі.
1. Проєктування бази даних і серверної архітектури.
1. Програмна реалізація серверної частини.
1. Тестування та перевірка програмної реалізації.

Висновки.

Список використаних джерел.

Додатки.

ГРАФІЧНА ЧАСТИНА

1. Структура сутностей бази даних.
2. Схема серверної архітектури.
3. Скріншоти Swagger/OpenAPI та live API-сценарію.

Дата видачі «__» _____ 2026 р.

Завдання видав _____ Кудрявцев Д. С.

Завдання отримав _____ Саволук М. М.

АНОТАЦІЯ

УДК _____

Саволук М. М. Розробка серверної частини інформаційної системи VocabMiner. Курсова робота з дисципліни «Алгоритмізація та програмування». Вінниця: ВНТУ, 2026. Українською мовою.

У курсовій роботі розроблено серверну частину інформаційної системи VocabMiner, призначеної для збереження навчального контенту, виділення словникових одиниць із контексту, формування пояснень, організації повторення та експорту словника. Робота орієнтована на демонстрацію повного серверного циклу: від проєктування бази даних до реалізації контролерів, сервісів, репозиторіїв і перевірки працездатності API.

Серверну частину реалізовано на ASP.NET Core 8 з використанням PostgreSQL, Entity Framework Core та Npgsql. Структура проєкту побудована за багаторівневим підходом: контролери приймають HTTP-запити, сервіси виконують бізнес-логіку, репозиторії інкапсулюють доступ до даних, а доменні моделі описують сутності системи. У роботі спроектовано 7 сутностей бази даних, реалізовано 6 контролерів і 105 HTTP endpoint-ів, підготовлено seed-дані, unit-тести та ручний HTTP-сценарій.

Практичну працездатність серверної частини підтверджено тестами бізнес-логіки та live-перевіркою з реальною PostgreSQL БД у Docker. Під час перевірки було застосовано міграцію Entity Framework Core, виконано імпорт контенту, створення сегментів, словникової одиниці, mock-пояснення, результату повторення та TSV-експорту.

ЗМІСТ

1.1 Призначення системи VocabMiner.....	1
1.2 Основні сценарії використання.....	1
1.3 Аналіз вимог до курсової реалізації.....	1
1.4 Постановка задачі.....	1
2.1 Вибір технологій.....	1
2.2 Проєктування сутностей бази даних.....	1
2.3 Зв'язки між сутностями.....	1
2.4 Серверна архітектура.....	1
2.5 Структура серверного проєкту.....	1
3.1 Реалізація контролерів.....	1
3.2 Реалізація моделей і ViewModels.....	1
3.3 Реалізація бізнес-логіки.....	1
3.4 Реалізація репозиторіїв.....	1

3.5 Інтеграція компонентів.....	1
4.1 Unit-тестування бізнес-логіки.....	1
4.2 Ручний HTTP-сценарій.....	1
4.3 Перевірка з реальною PostgreSQL БД.....	1
4.4 Результати перевірки.....	1
Додаток А. Повний перелік API-запитів і ViewModels.....	1
А.1. Повна таблиця API-запитів.....	1
А.2. ViewModels, що використовуються в API.....	1
Додаток Б. Приклад JSON-запиту імпорту контенту.....	1
Додаток В. Приклад JSON-запиту створення словникової одиниці.....	1
Додаток Г. Фрагмент доменної моделі.....	1
Додаток Д. Фрагмент бізнес-логіки повторення.....	1

ВСТУП

Сучасні програмні системи дедалі частіше будуються як набір взаємопов'язаних серверних компонентів, які отримують запити від клієнтських застосунків, працюють із базою даних, виконують бізнес-логіку та повертають структуровані відповіді. Такий підхід є основою клієнт-серверної архітектури й широко використовується у вебзастосунках, мобільних сервісах, освітніх платформах і персональних продуктах для роботи з даними.

Однією з прикладних задач, для яких доцільно застосовувати серверну архітектуру, є підтримка вивчення іноземної лексики з реального контенту. У традиційному підході слова часто вивчаються ізольовано, без речення або ситуації, у якій вони були використані. Через це користувачу складніше запам'ятати значення слова, його вживання та зв'язок із ширшим контекстом. Система VocabMiner розглядає словникову одиницю не як окремий запис, а як елемент, пов'язаний із джерелом контенту, сегментом тексту, поясненням, повторенням і можливістю експорту.

Актуальність роботи полягає в тому, що вона поєднує навчальну предметну область із типовими задачами серверної розробки: моделюванням даних,

реалізацією REST API, застосуванням dependency injection, розділенням відповідальності між рівнями системи та перевіркою працездатності з реальною базою даних. Такий проєкт дає змогу продемонструвати не лише окремі алгоритмічні або синтаксичні навички, а й здатність побудувати цілісний серверний застосунок.

Метою курсової роботи є розробка серверної частини інформаційної системи VocabMiner на основі клієнт-серверної архітектури з використанням ASP.NET Core 8, PostgreSQL та Entity Framework Core.

Об'єктом дослідження є процес побудови серверної частини інформаційної системи.

Предметом дослідження є архітектура, модель даних і програмна реалізація backend-системи для майнінгу словника з навчального контенту.

Для досягнення мети необхідно розв'язати такі завдання:

1. Проаналізувати предметну область системи VocabMiner і визначити основні сценарії використання.
2. Спроекувати реляційну базу даних PostgreSQL, пов'язану з темою курсової роботи.
3. Організувати серверний проєкт за рівнями Controllers, Business Logic, Repositories і Domains.
4. Реалізувати контролери, ViewModels, сервіси, репозиторії та dependency injection.
5. Підготувати тестові дані, unit-тести й ручний HTTP-сценарій.
6. Перевірити працездатність серверної частини з реальною PostgreSQL БД.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Призначення системи VocabMiner

VocabMiner призначений для користувачів, які вивчають іноземну мову через

реальний контент: субтитри, статті, діалоги, фрагменти відео або навчальні тексти. Основна ідея системи полягає в тому, щоб зберігати не лише слово або фразу, а й контекст, у якому ця словникова одиниця була знайдена. Такий підхід робить навчання ближчим до практичного використання мови, оскільки користувач бачить приклад уживання, пояснення та може повернутися до повторення пізніше.

У межах курсової роботи реалізовано серверну частину системи. Вона не містить повноцінного клієнтського інтерфейсу, але надає HTTP API, через який клієнтський застосунок може імпортувати текст, отримувати сегменти, створювати словникові одиниці, запускати пояснення, проходити повторення та створювати експорт. Таким чином, сервер виступає центральним рівнем, який зберігає стан системи та забезпечує доступ до нього через структуровані запити.

1.2 Основні сценарії використання

Типовий сценарій роботи починається з користувача, який має певний навчальний текст. Клієнтський застосунок надсилає цей текст на сервер, після чого сервер зберігає джерело контенту й автоматично розбиває його на сегменти. Сегментами можуть бути речення або короткі фрагменти, зручні для подальшого аналізу.

Після імпорту користувач може створити словникову одиницю на основі слова або фрази із сегмента. Система нормалізує термін, зберігає його, прив'язує до контексту й може сформувати mock-пояснення. У реальному продукті таке пояснення могло б створюватися за допомогою LLM API, але в курсовій реалізації використано mock-сервіс, щоб не залежати від зовнішніх сервісів і зосередитися на серверній архітектурі.

Далі користувач може переглядати список одиниць, проходити повторення, фіксувати результат і створювати експорт словника у CSV або TSV. Ці сценарії охоплюють основні операції над даними: створення, читання, оновлення, видалення, фільтрацію, формування похідних результатів і роботу з файлами експорту.

1.3 Аналіз вимог до курсової реалізації

Згідно з вимогами курсової роботи, серверний проєкт повинен містити базу даних, пов'язану з темою, не менше трьох сутностей із п'ятьма або більше полями, CRUD-операції, набір контролерів, ViewModels, сервіси бізнес-логіки, репозиторії та dependency injection. Окремою умовою було забезпечено розширений перелік endpoint-ів: у реалізованому проєкті кожен із шести контролерів має не менше 15 HTTP-запитів.

Тема VocabMiner добре відповідає цим вимогам, оскільки природно містить кілька пов'язаних сутностей: користувача, джерело контенту, сегмент, словникову одиницю, появу одиниці в тексті, спробу повторення та експорт. Кожна з цих сутностей має окрему роль у предметній області й може бути представлена таблицею реляційної бази даних.

1.4 Постановка задачі

Необхідно розробити серверну частину інформаційної системи VocabMiner, яка забезпечує збереження навчального контенту, поділ контенту на сегменти, створення словникових одиниць із контекстом, формування токс-пояснень, повторення матеріалу та експорт словника у файл. Сервер має працювати з PostgreSQL, використовувати Entity Framework Core для доступу до даних і надавати HTTP API для основних сценаріїв.

Результатом виконання роботи має бути інтегрований серверний застосунок, у якому контролери, сервіси, репозиторії та база даних пов'язані в єдиний програмний комплекс. Окрім коду, необхідно підготувати пояснювальну записку, ілюстративну частину, тестові дані та підтвердження працездатності системи.

2 ПРОЄКТУВАННЯ БАЗИ ДАНИХ І СЕРВЕРНОЇ АРХІТЕКТУРИ

2.1 Вибір технологій

Для реалізації серверної частини обрано ASP.NET Core 8, оскільки ця

платформа дає змогу створювати продуктивні Web API, має вбудовану підтримку dependency injection, middleware, конфігурації, логування та інтеграції зі Swagger/OpenAPI. Для зберігання даних використано PostgreSQL - реляційну СУБД, яка підтримує зовнішні ключі, індекси, транзакції та добре інтегрується з .NET через провайдер Npgsql.

Entity Framework Core використано як ORM-рівень між доменними класами та таблицями бази даних. Це дозволяє описати сутності як C#-класи, налаштувати зв'язки між ними та виконувати міграції схеми бази даних. Такий вибір відповідає навчальній меті роботи, оскільки демонструє зв'язок між моделлю предметної області, структурою бази даних і серверним кодом.

2.2 Проектування сутностей бази даних

У системі спроектовано сім основних сутностей. Їхній набір вибрано так, щоб охопити весь навчальний цикл: від імпорту контенту до повторення й експорту словника.

Сутність	Призначення
'User'	Зберігає дані користувача, мови навчання та службову інформацію профілю.
'ContentSource'	Описує імпортоване джерело навчального контенту.
'Segment'	Містить окремий фрагмент або речення з джерела контенту.
'LearningUnit'	Представляє слово або фразу, яку користувач вивчає.
'Occurrence'	Зв'язує словникову одиницю з конкретним сегментом тексту.
'ReviewAttempt'	Зберігає результат повторення словникової одиниці.
'ExportJob'	Описує створений експорт словника у CSV або TSV.

Кожна сутність містить щонайменше п'ять полів. Наприклад, 'LearningUnit' має ідентифікатор, зв'язок із користувачем, термін, нормалізований термін,

тип одиниці, переклад, пояснення, приклад речення, статус, складність, дату наступного повторення та службові часові поля. Така структура дозволяє зберігати не лише мінімальний словник, а й стан навчання.

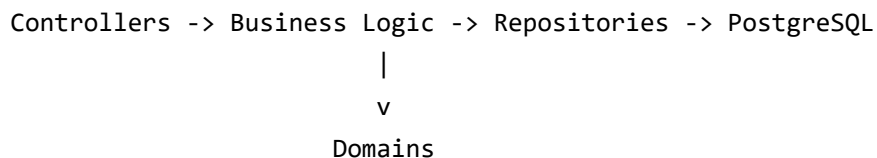
2.3 Зв'язки між сутностями

Основним власником даних у системі є користувач. Один користувач може мати багато джерел контенту, багато словникових одиниць, багато спроб повторення та багато експортів. Джерело контенту складається з багатьох сегментів. Словникова одиниця може зустрічатися в кількох сегментах, тому між `LearningUnit` і `Segment` використовується проміжна сутність `Occurrence`.

Повторення винесені в окрему таблицю `ReviewAttempt`, оскільки вони представляють історію навчальних дій. Експорт також оформлено окремою сутністю `ExportJob`, що дозволяє зберігати формат, статус, назву файлу, кількість рядків і час завершення формування файлу.

2.4 Серверна архітектура

Серверний проєкт побудовано за багаторівневою архітектурою. Контролери відповідають за приймання HTTP-запитів і повернення відповідей. Сервіси бізнес-логіки виконують основні правила предметної області. Репозиторії інкапсулюють доступ до бази даних. Доменні моделі та ViewModels описують структуру внутрішніх даних і форматів взаємодії з API.



Такий поділ зменшує зв'язаність між частинами системи. Контролер не повинен знати, як саме виконується SQL-запит, а репозиторій не повинен містити правила навчального процесу. Це спрощує тестування, підтримку й подальше розширення системи.

2.5 Структура серверного проекту

У проєкті створено папки, які відповідають вимогам курсової роботи та відображають архітектурні рівні системи.

Папка	Призначення
`Controllers`	HTTP-контролери серверного API.
`Business Logic`	Сервіси, які виконують бізнес-логіку.
`Repositories`	Інтерфейси та реалізації доступу до даних.
`Domains/Entities`	Доменні сутності, пов'язані з таблицями бази даних.
`Domains/ ViewModels`	Моделі запитів і відповідей API.
`Data`	DbContext і налаштування роботи з Entity Framework Core.
`Migrations`	Міграції схеми бази даних.

Dependency injection налаштовано в `Program.cs`. Через DI-контейнер реєструються DbContext, репозиторії та сервіси. Завдяки цьому контролери отримують залежності через конструктори, а не створюють їх самостійно.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ СЕРВЕРНОЇ ЧАСТИНИ

3.1 Реалізація контролерів

У серверній частині реалізовано шість контролерів. Їхній набір охоплює головні функціональні області системи: автентифікацію, роботу з джерелами контенту, сегментами, словниковими одиницями, повтореннями та експортом.

Контролер	Кількість endpoint-ів	Основне призначення
-----------	--------------------------	---------------------

`AuthController`	18	Реєстрація, авторизація, профіль користувача та допоміжні перевірки.
`ContentSourcesController`	17	Імпорт, перегляд, оновлення, видалення та аналіз джерел контенту.
`SegmentsController`	16	Отримання, пошук, редагування та контекстна навігація сегментами.
`LearningUnitsController`	20	Створення, оновлення, нормалізація, пояснення та керування словниковими одиницями.
`ReviewsController`	17	Черга повторення, історія, статистика та фіксація результатів.
`ExportsController`	17	Створення, перегляд, preview, завантаження та видалення експортів.
Разом	**105**	Повний набір HTTP-запитів серверної частини.

Повна таблиця API-запитів із параметрами, типами, request/response ViewModels і типами результатів наведена в додатку А. В основному тексті доцільно подати лише підсумок, щоб не перевантажувати розділ довгою технічною таблицею.



VocabMinerCourseWork.Api 1.0 OAS 3.0

<http://localhost:5152/swagger/v1/swagger.json>

Auth

POST /auth/register

POST /auth/login

GET /auth/profile/{id}

GET /auth/health

GET /auth/options/languages

GET /auth/options/defaults

POST /auth/check-email

POST /auth/validate-register

POST /auth/validate-login

POST /auth/profile/lookup

GET /auth/profile/by-email

GET /auth/profile/{id}/exists

GET /auth/profile/{id}/status

3.2 Реалізація моделей і ViewModels

Доменні моделі описують сутності бази даних і використовуються всередині серверної частини. Наприклад, `'ContentSource'` представляє імпортований текст, `'Segment'` - його фрагмент, а `'LearningUnit'` - слово або фразу для вивчення. ViewModels відокремлюють внутрішню структуру даних від HTTP-контракту й визначають, які поля клієнт передає або отримує у відповіді.

Такий поділ є важливим для підтримки API. Якщо внутрішня сутність містить службові поля або навігаційні властивості Entity Framework Core, їх не обов'язково передавати клієнту. Натомість ViewModel задає контрольований формат запиту або відповіді.

3.3 Реалізація бізнес-логіки

Бізнес-логіка винесена в сервіси. `'AuthService'` відповідає за реєстрацію й перевірку користувача. `'ContentImportService'` імпортує текст і виконує поділ на сегменти. `'LearningUnitService'` створює та оновлює словникові одиниці, нормалізує терміни й прив'язує їх до контексту. `'MockExplanationService'` формує навчальне пояснення без звернення до зовнішнього AI API. `'ReviewService'` обчислює наступну дату повторення, а `'ExportService'` формує CSV або TSV-вміст.

Наприклад, під час імпорту контенту клієнт надсилає текст, назву, тип джерела й мову. Контролер приймає запит і передає його в сервіс. Сервіс створює сутність джерела, розбиває текст на сегменти, готує пов'язані записи й передає їх до репозиторію для збереження в базі даних. Такий потік демонструє зв'язок між HTTP API, бізнес-логікою та рівнем даних.

3.4 Реалізація репозиторіїв

Репозиторії інкапсулюють роботу з Entity Framework Core. Вони виконують пошук, фільтрацію, створення, оновлення та видалення записів. Завдяки

цьому сервіси працюють із методами репозиторіїв і не залежать від конкретних LINQ-запитів або деталей DbContext.

Наприклад, репозиторій словникових одиниць відповідає за отримання одиниць користувача, пошук за статусом, пошук нормалізованого терміна та збереження змін. Репозиторій експортів зберігає інформацію про сформовані файли й дозволяє отримати їх за користувачем або ідентифікатором.

3.5 Інтеграція компонентів

Інтеграція серверної частини виконується в `Program.cs`. У цьому файлі налаштовуються контролери, JSON-серіалізація enum-типів, Swagger, підключення до PostgreSQL, DbContext, репозиторії та сервіси. Також передбачено можливість застосування міграцій під час запуску через параметр `ApplyMigrations`.

OpenAPI / Swagger зведення VocabMi

Згенеровано з live endpoint /swagger/v1/swagger.json о 2026-05-11 23:16:23

Контролери та штрафна умова

Контролер	Endpoint-и	15+
Auth	18	Так
ContentSources	17	Так
Exports	17	Так
LearningUnits	20	Так
Reviews	17	Так
Segments	16	Так

Додатково у Swagger є сервісний GET /, який не належить до контролерів.

Фрагмент п

Контролер
Auth
Auth
Auth
Auth
Auth
Auth
Auth
Auth
Auth
Auth
Auth
Auth

Рисунок 3.2 - Стислий огляд контролерів у Swagger/OpenAPI.

Після запуску сервер надає кореневий endpoint, який повертає службову відповідь зі статусом ``running``, і підключає всі контролери через ``MapControllers()``. У режимі розробки доступна сторінка Swagger, що дає змогу переглядати та вручну перевіряти API.

4 ТЕСТУВАННЯ ТА ПЕРЕВІРКА ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

4.1 Unit-тестування бізнес-логіки

Для перевірки серверної частини створено тестовий проєкт ``VocabMinerCourseWork.Tests``. Він містить тести, які перевіряють ключові частини бізнес-логіки: поділ тексту на сегменти, нормалізацію словникової одиниці, розрахунок наступної дати повторення та формування TSV-експорту.

Такі тести не замінюють повної інтеграційної перевірки, але підтверджують, що основні алгоритмічні частини серверної логіки працюють очікувано. Особливо важливо, що тестування охоплює ті операції, які не є простим збереженням даних: сегментацію, нормалізацію, планування повторення та побудову експортного вмісту.

4.2 Ручний HTTP-сценарій

Окремо підготовлено файл ``VocabMinerCourseWork.Api.http``, який дозволяє вручну пройти основний сценарій роботи системи. Сценарій включає авторизацію seed-користувача, імпорт контенту, перегляд сегментів, створення словникової одиниці, отримання mock-пояснення, проходження повторення, створення експорту та завантаження файлу.

Ручний сценарій важливий тому, що він перевіряє не окремий метод, а послідовну взаємодію кількох рівнів системи. Успішне проходження такого сценарію означає, що контролери, сервіси, репозиторії та база даних працюють узгоджено.

4.3 Перевірка з реальною PostgreSQL БД

Live-перевірку виконано 10 травня 2026 року з реальною PostgreSQL БД у Docker. Для запуску бази даних використано контейнер `vocabminer-coursework-postgres`. Після запуску Docker Compose контейнер перейшов у стан `healthy`, а API було запущено з параметром `ApplyMigrations=true`.

Під час старту сервер застосував міграцію Entity Framework Core `20260503183000\_InitialCreate`. Наявність міграції підтверджено через таблицю `\_\_EFMigrationsHistory`, у якій зафіксовано ProductVersion `8.0.0`.

Основні кроки live-сценарію наведено в таблиці 4.1.

Крок	Endpoint	Результат
1	`GET /`	API повернув статус `running`.
2	`POST /auth/login`	Виконано вхід seed-користувача `student@example.com`.
3	`POST /content-sources`	Створено нове джерело контенту.
4	`GET /segments/content-source/{id}`	Отримано сегменти імпортованого тексту.
5	`POST /learning-units`	Створено словникову одиницю.
6	`POST /learning-units/{id}/explain`	Отримано mock-пояснення.
7	`POST /reviews/submit`	Збережено результат повторення.
8	`POST /exports`	Створено TSV-експорт.
9	`GET /exports/{id}/download`	Файл експорту завантажено, HTTP 200.

Таблиця 4.1 - Основні кроки live-перевірки серверної частини.

Live API-сценарій VocabMiner

Перевірка виконана проти `http://localhost:5152` о 2026-05-11 23:14:51; PostgreSQL

8

content sources

3

learning units

HTTP	Endpoint
GET	/
POST	/auth/login
GET	/auth/profile/11111111-1111-1111-1111-111111111111
GET	/content-sources/count
GET	/learning-units/count
GET	/reviews/count-due
GET	/exports/user/{id}/count

Рисунок 4.1 - Live API-сценарій із реальною PostgreSQL БД.

4.4 Результати перевірки

Після виконання live-сценарію було підтверджено, що база даних створюється коректно, seed-дані застосовуються, міграція фіксується в історії EF Core, а сервер обробляє HTTP-запити з реальною PostgreSQL БД. Сценарій завершився успішним завантаженням TSV-файлу експорту.

Фінальна кількість записів у таблицях live PostgreSQL після виконання сценарію наведена в таблиці 4.2.

Таблиця	Кількість записів
`Users`	1
`ContentSources`	8
`Segments`	16
`LearningUnits`	3
`Occurrences`	2
`ReviewAttempts`	2
`ExportJobs`	2

Таблиця 4.2 - Кількість записів у таблицях після live-перевірки.

Отже, тестування підтвердило працездатність реалізованої серверної частини. Unit-тести перевіряють ключові правила бізнес-логіки, а live-сценарій демонструє роботу повного ланцюга `HTTP API -> сервіс -> репозиторій -> PostgreSQL`.

ВИСНОВКИ

У результаті виконання курсової роботи було розроблено серверну частину інформаційної системи VocabMiner. Система призначена для роботи з навчальним контентом, створення словникових одиниць із контекстом, формування тоск-пояснень, організації повторення та експорту словника у

CSV або TSV.

У першому розділі було проаналізовано предметну область і сформульовано задачу розробки. Було визначено, що VocabMiner повинен підтримувати повний серверний цикл роботи з навчальними даними: імпорт тексту, сегментацію, створення словникових одиниць, прив'язку до контексту, повторення й експорт.

У другому розділі було спроектовано базу даних і серверну архітектуру. Для зберігання даних обрано PostgreSQL, а для доступу до них - Entity Framework Core. У системі створено сім сутностей, кожна з яких має не менше п'яти полів і безпосередньо пов'язана з темою курсової роботи.

У третьому розділі було описано програмну реалізацію серверної частини. Проєкт організовано за рівнями `Controllers`, `Business Logic`, `Repositories` і `Domains`. Реалізовано шість контролерів, 105 HTTP endpoint-ів, ViewModels, сервіси, репозиторії та dependency injection. Такий підхід забезпечує розділення відповідальності між компонентами й полегшує підтримку системи.

У четвертому розділі було проведено тестування та перевірку реалізації. Unit-тести підтвердили роботу сегментації, нормалізації, планування повторення та експорту. Live-перевірка з реальною PostgreSQL БД підтвердила, що сервер застосовує міграції, працює з тестовими даними, обробляє HTTP-запити та виконує основний сценарій роботи VocabMiner.

Таким чином, поставлену мету досягнуто. Розроблена серверна частина відповідає вимогам курсової роботи, демонструє клієнт-серверну архітектуру, роботу з базою даних, CRUD-операції, бізнес-логіку, репозиторії, dependency injection і практичну перевірку працездатності.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Матеріали дисципліни «Алгоритмізація та програмування».
2. Приклад оформлення курсової роботи `Lebid\_1KN\_20b.pdf`.
3. Опис вимог до курсової роботи з дисципліни «Алгоритмізація та

програмування».

4. Microsoft Learn. ASP.NET Core Web API documentation.
5. Microsoft Learn. Entity Framework Core documentation.
6. PostgreSQL Documentation.
7. Npgsql Entity Framework Core Provider documentation.
8. Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

ДОДАТКИ

Додаток А. Повний перелік API-запитів і ViewModels

А.1. Повна таблиця API-запитів

У таблиці А.1 наведено базові HTTP-методи, параметри запитів, C#-типи, ViewModels та типи результатів, що повертаються методами контролерів. Службовий параметр `Cancellation token` використовується в асинхронних методах, але не є частиною HTTP-контракту, тому в таблиці не дублюється.

Контролер	Метод і шлях	Призначення	Параметри та типи	Результат
`AuthController`	`POST` `/auth/register`	Реєстрація користувача	`body`: `RegisterRequest`	`RegisterResponse`
`AuthController`	`POST` `/auth/login`	Авторизація користувача	`body`: `LoginRequest`	`LoginResponse`
`AuthController`	`GET` `/auth/profile/{id}`	Отримання профілю	`route id: Guid`	немає
`ContentSourcesController`	`GET` `/content-sources`	Список джерел користувача	`query` `userId: Guid`	немає
`ContentSourcesController`	`GET` `/content`	Отримання	`route id: Guid`	немає

	sources/{id}`	одного джерела	
`ContentSourcesController`	`POST /content-sources`	Імпорт нового контенту	`body: ContentSourceRequest`
`ContentSourcesController`	`PUT /content-sources/{id}`	Оновлення джерела	`route id: Guid`; `body: ContentSourceUpdateRequest`
`ContentSourcesController`	`DELETE /content-sources/{id}`	Видалення джерела	`route id: Guid` немає
`SegmentsController`	`GET /segments/content-source/{contentSourceId}`	Сегменти джерела	`route contentSourceId: Guid` немає
`SegmentsController`	`GET /segments/{id}`	Отримання сегмента	`route id: Guid` немає
`SegmentsController`	`PUT /segments/{id}`	Оновлення тексту сегмента	`route id: Guid`; `body: SegmentUpdateRequest`
`SegmentsController`	`DELETE /segments/{id}`	Видалення сегмента	`route id: Guid` немає
`LearningUnitsController`	`GET /learning-units`	Список словникових одиниць	`query userId: Guid`; `query status: LearningStatus?`; `query search:

			string?`
`LearningUnitsController`	`GET /learning-units/{id}`	Отримання словникової одиниці	`route id: Guid` немає
`LearningUnitsController`	`POST /learning-units`	Створення словникової одиниці	`body: LearningUnitReq `Learn uest`
`LearningUnitsController`	`PUT /learning-units/{id}`	Оновлення словникової одиниці	`route id: Guid`; `body: LearningUnitUpd `Learn ateRequest`
`LearningUnitsController`	`DELETE /learning-units/{id}`	Видалення словникової одиниці	`route id: Guid` немає
`LearningUnitsController`	`POST /learning-units/{id}/explain`	Mock-пояснення	`route id: Guid` немає
`LearningUnitsController`	`POST /learning-units/promote-phrase`	Створення фрази з сегмента	`body: PromotePhraseR `Prom equest`
`ReviewsController`	`GET /reviews/today/{userId}`	Список слів для повторення	`route Guid` `userId: немає
`ReviewsController`	`GET /reviews/history/{userId}`	Історія повторень	`route Guid` `userId: немає
`ReviewsController`	`POST /reviews/submit`	Збереження результату	`body: ReviewSubmitRe `Review

		повторення	quest`	
`ReviewsController`	`POST /reviews/reset/{learningUnitId}`	Скидання прогресу	`route learningUnitId: Guid`	немає
`ExportsController`	`POST /exports`	Створення експорту	`body: ExportCreateReq uest`	`Export`
`ExportsController`	`GET /exports/user/{userId}`	Список експортів	`route Guid`	userId: немає
`ExportsController`	`GET /exports/{id}`	Метадані експорту	`route id: Guid`	немає
`ExportsController`	`GET /exports/{id}/download`	Завантаження файлу	`route id: Guid`	немає
`ExportsController`	`DELETE /exports/{id}`	Видалення експорту	`route id: Guid`	немає

Для виконання штрафної умови про мінімум 15 запитів на кожен контролер API було розширено додатковими endpoint-ами. Після розширення кожен із 6 контролерів має не менше 15 HTTP-запитів.

Контролер	Кількість endpoint-ів
`AuthController`	18
`ContentSourcesController`	17
`SegmentsController`	16
`LearningUnitsController`	20

`ReviewsController`	17
`ExportsController`	17
Разом	**105**

Додаткові endpoint-и:

Контролер	Метод і шлях	Призначення	Параметри та типи	
`AuthController`	`GET /auth/health`	Перевірка доступності API	auth немає	немає
`AuthController`	`GET /auth/options/languages`	Доступні мовні опції	має немає	немає
`AuthController`	`GET /auth/options/default`	Значення за замовчуванням для реєстрації	має немає	немає
`AuthController`	`POST /auth/check-email`	Перевірка доступності email	`body: EmailAvailabilityRequest`	`Email`
`AuthController`	`POST /auth/validate-register`	Перевірка моделі реєстрації	`body: RegisterRequest`	`Register`
`AuthController`	`POST /auth/validate-login`	Перевірка моделі логіну	`body: LoginRequest`	`Login`
`AuthController`	`POST /auth/profile/lookup`	Пошук профілю за email	`body: ProfileLookupRequest`	`Profile`

`AuthController`	`GET` /auth/profile/by-email`	Отримання профілю за email	`query` email: string`	email: немає
`AuthController`	`GET` /auth/profile/{id}/exists`	Перевірка існування профілю	`route id: Guid`	немає
`AuthController`	`GET` /auth/profile/{id}/status`	Активність і дата входу	`route id: Guid`	немає
`AuthController`	`GET` /auth/profile/{id}/languages`	Мови користувача	`route id: Guid`	немає
`AuthController`	`GET` /auth/profile/{id}/audit`	Технічні дані профілю	`route id: Guid`	немає
`AuthController`	`POST` /auth/logout`	Mock-вихід із системи	із немає	немає
`AuthController`	`POST` /auth/refresh-session`	Mock-оновлення сесії	немає	немає
`AuthController`	`GET` /auth/schema`	Опис auth ViewModels	немає	немає
`ContentSourcesController`	`GET` /content-sources/recent`	Останні джерела	`query` Guid`; take: int`	userId: немає
`ContentSourcesController`	`GET` /content-sources/count`	Кількість джерел	`query` Guid`	userId: немає
`ContentSourcesController`	`GET` /content-	Пошук	у `query`	userId: немає

	sources/search`	джерелах	Guid`; `query term: string`	
`ContentSourcesController`	`GET /content-sources/by-type`	Фільтр за типом джерела	`query` `userId: Guid`; `query` `sourceType: ContentSourceType`	немає
`ContentSourcesController`	`GET /content-sources/languages`	Мови джерел користувача	`query` `userId: Guid`	немає
`ContentSourcesController`	`GET /content-sources/{id}/summary`	Зведення джерела	`route id: Guid`	немає
`ContentSourcesController`	`GET /content-sources/{id}/segments-count`	Кількість сегментів джерела	`route id: Guid`	немає
`ContentSourcesController`	`GET /content-sources/{id}/preview`	Текстовий прев'ю джерела	`route id: Guid`; `query` `length: int`	немає
`ContentSourcesController`	`GET /content-sources/{id}/exists`	Перевірка існування джерела	`route id: Guid`	немає
`ContentSourcesController`	`POST /content-sources/preview-segments`	Попередній поділ тексту	`body: TextSegmentationRequest`	немає
`ContentSourcesController`	`POST /content-sources/validate`	Перевірка моделі імпорту	`body: ContentSourceRequest`	немає
`ContentSourcesController`	`GET /content-`	Доступні типи	немає	немає

	sources/types`	джерел		
`SegmentsController`	`GET /segments/{id}/exists`	Перевірка існування сегмента	`route id: Guid`	немає
`SegmentsController`	`GET /segments/{id}/length`	Довжина сегмента	`route id: Guid`	немає
`SegmentsController`	`GET /segments/{id}/content`	Попередній і наступний сегмент	`route id: Guid`	немає
`SegmentsController`	`GET /segments/content-source/{contentSourceId}/count`	Кількість сегментів джерела	`route contentSourceId: Guid`	немає
`SegmentsController`	`GET /segments/content-source/{contentSourceId}/first`	Перший сегмент джерела	`route contentSourceId: Guid`	немає
`SegmentsController`	`GET /segments/content-source/{contentSourceId}/last`	Останній сегмент джерела	`route contentSourceId: Guid`	немає
`SegmentsController`	`GET /segments/content-source/{contentSourceId}/range`	Діапазон позицій	`route contentSourceId: Guid`; `query from: int`; `query to: int`	немає
`SegmentsController`	`GET /segments/content-`	Пошук	`route contentSourceId: Guid`	немає

	source/{contentSourceId}/search`	сегментах	Guid`; `query term: string`
`SegmentsController`	`GET` /segments/content-source/{contentSourceId}/positions`	Список позицій сегментів	`route` contentSourceId: немає Guid`
`SegmentsController`	`POST` /segments/split-preview`	Preview тексту	поділу `body:` TextSegmentationRequest`
`SegmentsController`	`POST` /segments/normalize-preview`	Preview нормалізації тексту	`body:` SegmentUpdateRequest`
`SegmentsController`	`GET` /segments/content-source/{contentSourceId}/empty`	Порожні сегменти	`route` contentSourceId: немає Guid`
`LearningUnitsController`	`GET` /learning-units/count`	Кількість словникових одиниць	`query` Guid`; `query` status: LearningStatus?`
`LearningUnitsController`	`GET` /learning-units/due`	Одиниці, що вже потребують повторення	`query` Guid`
`LearningUnitsController`	`GET` /learning-units/known`	Відомі одиниці	`query` Guid`
`LearningUnitsController`	`GET` /learning-units/new`	Нові одиниці	`query` Guid`
`LearningUnitsController`	`GET` /learning-	Засвоєні одиниці	`query` Guid`

	units/mastered`		Guid`	
`LearningUnitsController`	`GET /learning-units/ignored`	Проігноровані одиниці	`query` `userId: Guid`	немає
`LearningUnitsController`	`GET /learning-units/search`	Пошук одиниць	`query` `userId: Guid`; `query` `term: string`	немає
`LearningUnitsController`	`GET /learning-units/{id}/occurrences-count`	Кількість появи одиниці	`route id: Guid`	немає
`LearningUnitsController`	`GET /learning-units/{id}/review-state`	Стан повторення	`route id: Guid`	немає
`LearningUnitsController`	`POST /learning-units/normalize`	Нормалізація терміна	`body`: LearningUnitNormalizeRequest`	`LearningUnitNormalizeRequest`
`LearningUnitsController`	`POST /learning-units/{id}/mark-known`	Позначити відоме	як `route id: Guid`	немає
`LearningUnitsController`	`POST /learning-units/{id}/mark-ignored`	Позначити проігнороване	як `route id: Guid`	немає
`LearningUnitsController`	`POST /learning-units/{id}/mark-learning`	Позначити як у процесі вивчення	як у `route id: Guid`	немає
`ReviewsController`	`GET /reviews/queue/{userId}`	Черга повторення	`route` `Guid`	`userId`: немає
`ReviewsController`	`GET`	Кількість due-	`route`	`userId`: немає

	/reviews/count- due/{userId}`	карток		Guid`	
`ReviewsController`	`GET /reviews/history/{u serId}/latest`	Останні спроби повторення		`route Guid`; take: int`	userId: `query немає`
`ReviewsController`	`GET /reviews/history/{u serId}/count`	Кількість спроб повторення		`route Guid`	userId: немає
`ReviewsController`	`GET /reviews/stats/{user Id}`	Статистика повторень		`route Guid`	userId: немає
`ReviewsController`	`GET /reviews/grades/opt ions`	Доступні оцінки повторення		немає	немає
`ReviewsController`	`GET /reviews/next/{lear ningUnitId}`	Наступне повторення одиниці		`route learningUnitId: Guid`	немає
`ReviewsController`	`GET /reviews/{learning UnitId}/status`	Review-статус одиниці		`route learningUnitId: Guid`	немає
`ReviewsController`	`POST /reviews/submit- again`	Швидка оцінка `Again`		`body: ReviewQuickSub mitRequest`	`ReviewQuickSub mitRequest`
`ReviewsController`	`POST /reviews/submit- hard`	Швидка оцінка `Hard`		`body: ReviewQuickSub mitRequest`	`ReviewQuickSub mitRequest`
`ReviewsController`	`POST /reviews/submit-`	Швидка оцінка `Good`		`body: ReviewQuickSub`	`ReviewQuickSub`

	good`			mitRequest`
`ReviewsController`	`POST /reviews/submit-easy`	Швидка оцінка `Easy`		`body: ReviewQuickSubmitRequest`
`ReviewsController`	`POST /reviews/reset-today/{userId}`	Скидання карток користувача	due-	`route userId: Guid` немає
`ExportsController`	`POST /exports/csv`	Створення експорту	CSV-	`body: ExportCreateRequest`
`ExportsController`	`POST /exports/tsv`	Створення експорту	TSV-	`body: ExportCreateRequest`
`ExportsController`	`GET /exports/user/{userId}/latest`	Останні екпорти		`route userId: Guid`; `query take: int` немає
`ExportsController`	`GET /exports/user/{userId}/count`	Кількість експортів		`route userId: Guid` немає
`ExportsController`	`GET /exports/user/{userId}/completed`	Завершені екпорти		`route userId: Guid` немає
`ExportsController`	`GET /exports/{id}/exists`	Перевірка існування експорту		`route id: Guid` немає
`ExportsController`	`GET /exports/{id}/meta-data`	Метадані файлу експорту		`route id: Guid` немає

`ExportsController`	`GET` /exports/{id}/preview`	Preview перших рядків файлу	`route id: Guid`; `query lines: int`	немає
`ExportsController`	`GET` /exports/{id}/content-type`	MIME-тип експорту	`route id: Guid`	немає
`ExportsController`	`GET` /exports/formats`	Доступні формати	немає	немає
`ExportsController`	`GET` /exports/statuses`	Доступні статуси export job	немає	немає
`ExportsController`	`POST` /exports/validate`	Перевірка моделі експорту	`body: ExportCreateRequest`	немає

A.2. ViewModels, що використовуються в API

ViewModel	Тип	Поля та C#-типи
`RegisterRequest`	Request	`Email: string`; `DisplayName: string`; `Password: string`; `NativeLanguage: string`; `TargetLanguage: string`
`LoginRequest`	Request	`Email: string`; `Password: string`
`UserResponse`	Response	`Id: Guid`; `Email: string`; `DisplayName: string`; `NativeLanguage: string`; `TargetLanguage: string`; `CreatedAt: DateTime`; `LastLoginAt: DateTime?`;

		`IsActive: bool`
		`UserId: Guid`; `Title: string`;
		`SourceType: ContentSourceType`;
`ContentSourceRequest`	Request	`OriginalText: string`;
		`Language: string`; `Notes: string?`
		`Title: string`; `SourceType: ContentSourceType`;
`ContentSourceUpdateRequest`	Request	`OriginalText: string`;
		`Language: string`; `Notes: string?`
		`Id: Guid`; `UserId: Guid`; `Title: string`;
		`SourceType: ContentSourceType`;
`ContentSourceResponse`	Response	`OriginalText: string`;
		`Language: string`; `ImportedAt: DateTime`; `SegmentCount: int`;
		`Notes: string?`
`SegmentUpdateRequest`	Request	`Text: string`
		`Id: Guid`; `ContentSourceId: Guid`; `Position: int`; `Text: string`;
`SegmentResponse`	Response	`StartTime: TimeSpan?`; `EndTime: TimeSpan?`;
		`CreatedAt: DateTime`
		`UserId: Guid`; `Term: string`;
`LearningUnitRequest`	Request	`UnitType: LearningUnitType`;
		`Translation: string?`; `Explanation: string?`;

		`SegmentId: Guid?`
`LearningUnitUpdateRequest`	Request	`Term: string`; `UnitType: LearningUnitType`; `Translation: string?`; `Explanation: string?`; `ExampleSentence: string?`; `Status: LearningStatus`; `Difficulty: int`
`PromotePhraseRequest`	Request	`UserId: Guid`; `SegmentId: Guid`; `Phrase: string`
`LearningUnitResponse`	Response	`Id: Guid`; `UserId: Guid`; `Term: string`; `NormalizedTerm: string`; `UnitType: LearningUnitType`; `Translation: string?`; `Explanation: string?`; `ExampleSentence: string?`; `Status: LearningStatus`; `Difficulty: int`; `ReviewDueAt: DateTime`; `CreatedAt: DateTime`; `UpdatedAt: DateTime`; `OccurrenceCount: int`
`ExplanationResponse`	Response	`LearningUnitId: Guid`; `Term: string`; `Translation: string`; `Explanation: string`; `ExampleSentence: string`
`ReviewSubmitRequest`	Request	`UserId: Guid`; `LearningUnitId: Guid`; `Grade: ReviewGrade`; `ResponseTimeSeconds: int`; `Notes: string?`

`ReviewResponse`	Response	`LearningUnitId: Guid`; `Term: string`; `Translation: string?`; `Explanation: string?`; `ExampleSentence: string?`; `Status: LearningStatus`; `ReviewDueAt: DateTime`
`ReviewAttemptResponse`	Response	`Id: Guid`; `LearningUnitId: Guid`; `Grade: ReviewGrade`; `ResponseTimeSeconds: int`; `ReviewedAt: DateTime`; `NextDueAt: DateTime`; `Notes: string?`
`ExportCreateRequest`	Request	`UserId: Guid`; `Format: ExportFormat`; `FilterStatus: LearningStatus?`
`ExportJobResponse`	Response	`Id: Guid`; `UserId: Guid`; `Format: ExportFormat`; `FileName: string`; `Status: ExportStatus`; `FilterStatus: LearningStatus?`; `CreatedAt: DateTime`; `CompletedAt: DateTime?`; `RowCount: int`
`ApiMessageResponse`	Response	`Message: string`; `GeneratedAt: DateTime`
`CountResponse`	Response	`Scope: string`; `Count: int`
`ExistsResponse`	Response	`Id: Guid`; `Exists: bool`
`ValidationResponse`	Response	`IsValid: bool`; `Errors: IReadOnlyList<string>`
`EnumOptionResponse`	Response	`Name: string`; `Value: int`

`EmailAvailabilityRequest`	Request	`Email: string`
`EmailAvailabilityResponse`	Response	`Email: string`; `IsAvailable: bool`
`ProfileLookupRequest`	Request	`Email: string`
`RegisterDefaultsResponse`	Response	`NativeLanguage: string`; `TargetLanguage: string`; `MinPasswordLength: int`
`UserStatusResponse`	Response	`UserId: Guid`; `IsActive: bool`; `LastLoginAt: DateTime?`
`UserLanguageResponse`	Response	`UserId: Guid`; `NativeLanguage: string`; `TargetLanguage: string`
`UserAuditResponse`	Response	`UserId: Guid`; `CreatedAt: DateTime`; `LastLoginAt: DateTime?`; `IsActive: bool`
`TextSegmentationRequest`	Request	`Text: string`; `MaxSegments: int`
`SegmentPreviewResponse`	Response	`SegmentCount: int`; `Segments: IReadOnlyList<string>`
`ContentSourceSummaryResponse`	Response	`Id: Guid`; `Title: string`; `SourceType: ContentSourceType`; `Language: string`; `SegmentCount: int`; `ImportedAt: DateTime`; `TextLength: int`
`TextPreviewResponse`	Response	`Id: Guid`; `Preview: string`;

		`Length: int`
`SegmentContextResponse`	Response	`SegmentId: Guid`; `Position: int`; `PreviousText: string?`; `CurrentText: string`; `NextText: string?`
`SegmentLengthResponse`	Response	`SegmentId: Guid`; `CharacterCount: int`; `WordCount: int`
`SegmentPositionsResponse`	Response	`ContentSourceId: Guid`; `Positions: IReadOnlyList<int>`
`NormalizedTextResponse`	Response	`OriginalText: string`; `NormalizedText: string`
`LearningUnitNormalizeRequest`	Request	`Term: string`
`LearningUnitNormalizeResponse`	Response	`Term: string`; `NormalizedTerm: string`
`LearningUnitReviewStateResponse`	Response	`LearningUnitId: Guid`; `Status: LearningStatus`; `Difficulty: int`; `ReviewDueAt: DateTime`; `IsDue: bool`
`ReviewQuickSubmitRequest`	Request	`UserId: Guid`; `LearningUnitId: Guid`; `ResponseTimeSeconds: int`; `Notes: string?`
`ReviewStatsResponse`	Response	`UserId: Guid`; `DueCount: int`; `HistoryCount: int`; `AgainCount: int`; `HardCount: int`; `GoodCount: int`; `EasyCount: int`
`ExportMetadataResponse`	Response	`Id: Guid`; `FileName: string`;

```

        `Format: ExportFormat`; `Status:
        ExportStatus`; `RowCount: int`;
        `CreatedAt: DateTime`;
        `CompletedAt: DateTime?`;
        `ContentType: string`

`ExportPreviewResponse`           Response `Id: Guid`; `FileName: string`;
        `Lines: IReadOnlyList<string>`

```

У запитах використовуються такі enum-типи: `ContentSourceType`, `LearningUnitType`, `LearningStatus`, `ReviewGrade`, `ExportFormat`, `ExportStatus`. Вони зберігаються в базі як рядкові значення через налаштування Entity Framework Core.

Додаток Б. Приклад JSON-запиту імпорту контенту

```

{
  "userId": "11111111-1111-1111-1111-111111111111",
  "title": "Sample dialogue",
  "sourceType": "PlainText",
  "originalText": "I want to learn vocabulary from real content.\nContext makes
every phrase easier to remember.",
  "language": "en",
  "notes": "Manual test import"
}

```

Додаток В. Приклад JSON-запиту створення словникової одиниці

```

{
  "userId": "11111111-1111-1111-1111-111111111111",
  "term": "context",
  "unitType": "Word",
  "segmentId": "33333333-3333-3333-3333-333333333331"
}

```

Додаток Г. Фрагмент доменної моделі

```

public class LearningUnit

```



```
{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public string Term { get; set; } = string.Empty;
    public string NormalizedTerm { get; set; } = string.Empty;
    public LearningUnitType UnitType { get; set; }
    public string? Translation { get; set; }
    public string? Explanation { get; set; }
    public LearningStatus Status { get; set; }
    public DateTime ReviewDueAt { get; set; }
}
```

Додаток Д. Фрагмент бізнес-логіки повторення

```
public static DateTime CalculateNextDueAt(DateTime reviewedAt, ReviewGrade grade,
int difficulty)
{
    var days = grade switch
    {
        ReviewGrade.Again => 0,
        ReviewGrade.Hard => 1,
        ReviewGrade.Good => Math.Max(2, 4 - difficulty),
        ReviewGrade.Easy => Math.Max(4, 7 - difficulty),
        _ => 1
    };

    return reviewedAt.AddDays(days);
}
```